Supplementary Empirical Methods

Empirical Asset Pricing via Machine Learning

1 Estimation Framework

Presentation reference: Estimation Framework.

This note follows the methods and symbols in the supplied slides: $r_{i,t+1}$ is next-month excess return, $z_{i,t}$ is the predictor vector, and g is the expected-return function. Other symbols are defined within each method.

$$r_{i,t+1} = g(z_{i,t}) + \epsilon_{i,t+1}, \qquad \mathbb{E}[\epsilon_{i,t+1} \mid z_{i,t}] = 0. \tag{1.1}$$

The objective is to estimate $g(z_{i,t}) = \mathbb{E}[r_{i,t+1} \mid z_{i,t}]$. Squared-error prediction targets this conditional mean because, for any forecast $f(z)$,

$$\mathbb{E}[(r - f(z))^2 \mid z] = \mathrm{Var}(r \mid z) + \big(\mathbb{E}[r \mid z] - f(z)\big)^2. \tag{1.2}$$

Let $\mathcal{D}_{\text{train}}$ contain n training stock-month observations. Unless a different loss is stated,

$$\mathrm{MSE}(\theta) = \frac{1}{n} \sum_{(i,t)\in\mathcal{D}_{\text{train}}} \big[r_{i,t+1} - g(z_{i,t};\theta)\big]^2. \tag{1.3}$$

The models pool information across stocks and dates. A common function is estimated within each training window and reestimated annually.

Predictors and information timing

The $P = 920$ inputs consist of 94 stock characteristics, their 94×8 interactions with macroeconomic variables, and 74 industry indicators. Thus, even a model linear in z includes these *prespecified* characteristic–macro interactions. Accounting information is lagged to reflect availability; characteristics are ranked cross-sectionally and mapped to $[-1, 1]$.

Training, validation, and testing

Training estimates parameters; validation selects model complexity; testing evaluates forecasts using observations used for neither task.

Test year	Training years	Validation years	Test years
1987	1957–1974	1975–1986	1987
1988	1957–1975	1976–1987	1988

Training expands annually; validation remains a rolling twelve-year window. Test forecasts cover 1987–2016. Parameters use training data alone. Validation performance is not truly out of sample because validation selects hyperparameters. *Source: [1, Secs. 1.1 and 2.1]; [2, Appendix D].*

2 OLS, OLS-3, and Huber Loss

Presentation reference: Linear Forecasting Methods: OLS and Elastic Net; +H in the results.

OLS and OLS-3

Both use the linear forecasting function

$$g(z_{i,t};\theta) = z_{i,t}^{\top}\theta, \qquad \widehat{\theta}_{\text{OLS}} = \arg\min_{\theta} \text{MSE}(\theta). \tag{2.1}$$

For compactness, these expressions follow the slides and suppress a separate intercept. Centering or an explicit constant can handle an intercept without changing the distinction between the methods.

OLS uses all 920 inputs. **OLS-3** restricts the predictor set in advance to size, book-to-market, and momentum. It does not select three variables from the full set using the test sample.

Stack training returns into R and predictor rows into Z. Under full column rank, the unpenalized squared-error solution is

$$\widehat{\theta} = (Z^{\top}Z)^{-1}Z^{\top}R.$$

Rank-deficient problems require a suitable least-squares solution rather than this inverse.

Why can a large OLS model forecast poorly?

More inputs create more coefficients to estimate. Weak return predictability, redundant predictors, and dependence among stock returns can make estimates unstable. A large number of stock-month observations does not imply an equally large number of independent observations. OLS-3 limits complexity through a fixed, small predictor set.

Huber loss and the +H suffix

Let $e_{i,t+1} = r_{i,t+1} - g(z_{i,t};\theta)$. GKX's Huber loss is

$$H(e;\xi) = \begin{cases} e^2, & |e| \le \xi, \\ 2\xi|e| - \xi^2, & |e| > \xi, \end{cases} \qquad \xi > 0, \tag{2.2}$$

with training objective

$$\text{MSE}_H(\theta;\xi) = \frac{1}{n}\sum_{(i,t)\in\mathcal{D}_{\text{train}}} H(e_{i,t+1};\xi). \tag{2.3}$$

Small residuals receive squared loss; large residuals receive a loss growing linearly in magnitude. This reduces their influence on fitting. It does not remove observations or eliminate the need to control model complexity.

The conditional-mean identity in Section 1 applies to squared loss. Huber loss is a robust surrogate; its population optimum need not equal the conditional mean without additional conditions.

In the reported results, **OLS+H** and **OLS-3+H** replace squared loss with Huber loss. The same suffix also appears for ENet, GLM, and GBRT. These robust fits do not generally have the closed-form OLS solution above.

Threshold detail. Appendix Table A.5 describes ξ as a 99.9% quantile, but does not fully specify the underlying quantity used to construct that cutoff. No additional cutoff construction is assumed here. *Source: [1, Secs. 1.2–1.2.1; Table 1]; [2, Appendix B.1; Table A.5].*

3 Elastic Net: Penalization for Better OOS Prediction

Presentation reference: Linear Forecasting Methods: OLS and Elastic Net.

Model and objective

The forecasting function remains linear in the supplied predictors:

$$g(z_{i,t};\theta) = z_{i,t}^{\top}\theta. \tag{3.1}$$

Elastic Net changes how the coefficients are estimated:

$$\widehat{\theta}_{\text{ENet}} = \arg\min_{\theta} \left\{ \text{MSE}(\theta) + \lambda \left[(1-\rho) \sum_{j=1}^{P} |\theta_j| + \frac{\rho}{2} \sum_{j=1}^{P} \theta_j^2 \right] \right\}. \tag{3.2}$$

For **ENet+H**, replace MSE with MSE_H. The penalty has the same form.

What do the two penalty components do?

- The absolute-value component encourages some coefficients to become exactly zero, providing variable selection.
- The squared-coefficient component shrinks coefficients and can stabilize estimates when predictors are correlated.

The model accepts some bias in estimated coefficients in exchange for potentially lower estimation variance and better prediction on new observations.

Hyperparameters and the slide convention

Here $\lambda \geq 0$ controls the overall strength of penalization, and $0 \leq \rho \leq 1$ controls the balance:

Setting	Penalty under the slides' definition
$\lambda = 0$	No coefficient penalty
$\rho = 0$	Absolute-value penalty: lasso endpoint
$\rho = 1$	Squared-coefficient penalty: ridge endpoint
$0 < \rho < 1$	Combination of selection and shrinkage

Some software uses the opposite mixing convention, so the equation determines the meaning of ρ.

GKX implementation. The general model has both λ and ρ. Appendix Table A.5 fixes $\rho = 0.5$ and reports the penalty range $\lambda \in (10^{-4}, 10^{-1})$. This printed range should not be interpreted as a fully specified numerical grid.

Estimation

GKX use accelerated proximal gradient optimization. A gradient step reduces forecast loss; a proximal step applies the coefficient penalty; a momentum adjustment accelerates the iterations. This accommodates both squared and Huber losses.

Elastic Net changes coefficient estimation, not the functional form. Any characteristic–macro interactions it uses are already contained in z. *Source: [1, Sec. 1.3]; [2, Appendix B.1; Table A.5].*

4 Dimension Reduction: PCR and PLS

Presentation reference: Linear Forecasting Methods: Dimension Reduction.

Both methods replace P predictors with K linear combinations:

$$g(z_{i,t};\theta_K,\Omega_K) = z_{i,t}^\top \Omega_K \theta_K, \qquad \Omega_K = (w_1,\ldots,w_K) \in \mathbb{R}^{P\times K}. \tag{4.1}$$

Thus $Z\Omega_K$ is an $n \times K$ training score matrix. The number of retained components, K, controls complexity.

Principal Components Regression (PCR)

PCR finds directions preserving the greatest variation in the predictors. For component j,

$$\widehat{w}_{j,\text{PCR}} = \arg\max_w \text{Var}(Zw), \tag{4.2}$$

subject to $w^\top w = 1$ and $\text{Cov}(Zw, Zw_\ell) = 0$ for $\ell < j$. Covariances use the training observations, with the associated centering understood. Leading directions can be obtained by singular value decomposition.

Key point: returns do not enter the component-construction objective. High predictor variance need not imply strong return predictability.

Partial Least Squares (PLS)

PLS uses the training returns when constructing components:

$$\widehat{w}_{j,\text{PLS}} = \arg\max_w \text{Cov}^2(R, Zw), \tag{4.3}$$

under the same normalization and orthogonality constraints. Successive directions capture predictive covariation not represented by earlier components. GKX discuss the SIMPLS algorithm.

Key point: PLS seeks directions associated with the forecasting target, even when those directions do not explain the most predictor variation.

The regression step shared by both methods

Given the estimated loadings,

$$\widehat{\theta}_K = \arg\min_{\theta_K} \frac{1}{n}\left\|R - Z\widehat{\Omega}_K\theta_K\right\|_2^2. \tag{4.4}$$

Validation selects K. New observations use the loadings and preprocessing estimated from training data; test returns are not used to construct components.

	PCR	PLS
Direction criterion	Predictor variance	Covariation with returns
Uses training returns to construct directions?	No	Yes
Forecasting form	Linear in z	Linear in z

Combining correlated signals can average out noise. Retaining too many components can reintroduce estimation noise. These statistical components are not automatically economically identified asset-pricing factors. *Source: [1, Sec. 1.4, Eqs. (9)–(12)].*

5 Generalized Linear Model: Nonlinear Effects of Individual Predictors

Presentation reference: Generalized Linear Model.

Following the terminology in the slides and GKX, *GLM* here denotes an additive spline regression: it introduces nonlinear functions of the predictors into an otherwise linear-in-coefficients regression.

Model

Replace each input $z_{i,t,j}$ with a vector of spline basis functions:

$$p(z_{i,t,j}) = \left(p_1(z_{i,t,j}), \ldots, p_K(z_{i,t,j})\right)^\top.$$

Then

$$g(z_{i,t};\theta) = \sum_{j=1}^{P} p(z_{i,t,j})^\top \theta_j, \qquad \theta_j \in \mathbb{R}^K. \tag{5.1}$$

The basis functions allow the effect of an input to bend as its value changes. Knots mark where polynomial pieces join; second-order splines use quadratic pieces. The model is nonlinear in the original inputs but linear in the spline coefficients.

Group-lasso objective

$$\widehat{\theta}_{\text{GLM}} = \arg\min_{\theta} \left\{ \text{MSE}(\theta) + \lambda \sum_{j=1}^{P} \left(\sum_{k=1}^{K} \theta_{j,k}^2 \right)^{1/2} \right\}. \tag{5.2}$$

For **GLM+H**, replace MSE with MSE_H.

Each group contains the spline coefficients belonging to one input. Group lasso can set an entire coefficient vector θ_j to zero, removing that input's spline contribution. An active group may still have individual zero coefficients; the selection decision operates at group level.

What it captures

GLM allows an input to have, for example, a stronger marginal association with returns at low values than at high values. It adds these fitted functions across inputs. It does not directly learn unrestricted new interactions between different input coordinates.

The distinction concerns the *constructed inputs*: characteristic–macro interactions already supplied in z remain available to GLM.

Hyperparameters and the empirical specification

In the general slide formula, λ controls shrinkage and K determines the number of basis terms. K is not the same object as the number of spline knots.

GKX describe second-order splines. Their empirical Appendix Table A.5 fixes the number of knots at three and reports $\lambda \in (10^{-4}, 10^{-1})$. Thus the empirical specification is more specific than the slide's general statement about selecting spline complexity. Estimation uses accelerated proximal gradient optimization.

Why compare it with trees and neural networks? GLM introduces additive nonlinear effects. The comparison helps assess the additional value of methods that can learn conditional interactions. *Source: [1, Secs. 1.5–1.6]; [2, Appendix B.1; Table A.5].*

6 Random Forests (RF): Average Many Trees

Presentation reference: Tree Methods: Random Forests and GBRT.

The regression-tree building block

A tree divides predictor space into terminal regions C_k and assigns a constant prediction to each:

$$T(z;\theta) = \sum_{k=1}^{J} \theta_k \, \mathbf{1}\{z \in C_k\}, \qquad \widehat{\theta}_k = \frac{1}{|C_k|} \sum_{(i,t):\, z_{i,t} \in C_k} r_{i,t+1}. \tag{6.1}$$

Here $|C_k|$ counts observations in the tree's fitting sample, J denotes the number of terminal regions, and L denotes maximum depth. The symbol J distinguishes tree leaves from the K used for PCR/PLS or splines.

At each node, candidate splits divide observations according to one predictor and one threshold. Squared-error CART chooses the split giving the smallest combined within-child squared error. It is a greedy local search, not a global search over every possible tree.

Sequential splits produce interactions: a split on momentum inside a small-firm branch can differ from a split inside a large-firm branch.

Forest construction

For $b = 1, \ldots, B$:

1. Draw a bootstrap sample $\mathcal{D}_b$ from the training observations.
2. Grow a regression tree on that sample.
3. At each split, consider a random subset of m predictors rather than all P.

The slide summarizes each tree fit as

$$\widehat{\theta}_b \approx \arg\min_{\theta_b} \mathrm{MSE}_{\mathcal{D}_b}(\theta_b). \tag{6.2}$$

The forest prediction averages the fitted trees:

$$g(z;\Theta) = \frac{1}{B} \sum_{b=1}^{B} T_b(z;\widehat{\theta}_b). \tag{6.3}$$

Why the averaging helps

Individual trees can be unstable. Averaging reduces prediction variation, while random predictor subsets reduce similarity among trees. Tree depth must still be controlled; averaging alone does not remove every source of overfitting.

Hyperparameters

B is the number of trees, L their maximum depth, and m the number of candidate predictors per split. GKX's empirical specification uses 300 trees, depths from one to six, and varies the candidate-predictor count. The appendix prints that count's grid with an ellipsis, so no complete grid is reconstructed here.

RF trees can be fit independently. Their targets are returns in bootstrap samples, rather than residuals from previously fitted trees. *Source: [1, Sec. 1.6]; [2, Algorithms 2–3; Table A.5].*

7 GBRT: Sequentially Correct Forecast Errors

Presentation reference: Tree Methods: Random Forests and GBRT.

Gradient boosted regression trees build an additive forecast sequentially. With ν denoting the learning rate,

$$g_b(z) = g_{b-1}(z) + \nu T_b(z; \widehat{\theta}_b), \qquad b = 1, \ldots, B. \tag{7.1}$$

GKX's appendix initializes $g_0(z) = 0$.

Squared-error version shown in the slides

At step b, calculate the current training residuals

$$e_{i,t+1}^{(b-1)} = r_{i,t+1} - g_{b-1}(z_{i,t}).$$

Fit a shallow tree to those residuals:

$$\widehat{\theta}_b \approx \arg\min_{\theta_b} \frac{1}{n} \sum_{(i,t)\in\mathcal{D}_{\text{train}}} \left[e_{i,t+1}^{(b-1)} - T_b(z_{i,t}; \theta_b) \right]^2. \tag{7.2}$$

The next tree addresses errors left by the current ensemble. Shrinking its contribution by ν slows fitting and gives validation more control over complexity.

Why gradient boosting?

For a differentiable loss $\ell(r, g)$, form pseudo-responses

$$u_{i,t+1}^{(b)} = - \left. \frac{\partial \ell(r_{i,t+1}, g)}{\partial g} \right|_{g=g_{b-1}(z_{i,t})}. \tag{7.3}$$

The next tree approximates these negative gradients.

For half-squared loss, $\ell(r, g) = \frac{1}{2}(r - g)^2$, the pseudo-response is exactly the residual. For half-scaled Huber loss, it is

$$u_{i,t+1}^{(b)} = \text{clip}\left(e_{i,t+1}^{(b-1)}, -\xi, \xi \right), \tag{7.4}$$

where $\text{clip}(e, -\xi, \xi) = \max(-\xi, \min(e, \xi))$.

Thus the raw-residual equation in the slide explains squared-error boosting. The reported **GBRT+H** uses the Huber-loss gradient. A common scaling of the loss must be accompanied by a consistent learning-rate convention.

Hyperparameters and the distinction from RF

B controls boosting steps, L controls individual-tree depth, and ν controls the contribution of each new tree. Appendix Table A.5 reports depths one to two, up to 1,000 trees, and learning rates $\{0.01, 0.1\}$.

RF averages trees trained on resampled returns; GBRT adds trees trained to improve the current forecast. Boosted trees therefore depend on earlier fitting steps. *Source: [1, Sec. 1.6]; [2, Algorithm 4; Table A.5].*

8 Neural Networks

Presentation reference: Neural Networks.

Model construction

The slides describe a fully connected feed-forward network:

$$h^{(0)} = z_{i,t}, \tag{8.1}$$

$$h^{(\ell)} = \mathrm{ReLU}\left(W_\ell h^{(\ell-1)} + b_\ell\right), \qquad \ell = 1, \ldots, L, \tag{8.2}$$

$$g(z_{i,t}; \theta) = w_{\text{out}}^\top h^{(L)} + b_{\text{out}}. \tag{8.3}$$

W_ℓ contains connection weights and b_ℓ contains biases. ReLU acts componentwise as $\mathrm{ReLU}(x) = \max(0, x)$. The final output is linear and can be positive or negative, appropriate for an excess-return forecast.

Combining weighted inputs, nonlinear activations, and successive layers allows interactions to emerge during estimation. Removing every nonlinear activation would collapse the composition to an affine function.

Loss and optimization

$$\widehat{\theta}_{\text{NN}} \approx \arg\min_\theta \left\{ \mathrm{MSE}(\theta) + \lambda \sum_q |\theta_q| \right\}, \tag{8.4}$$

where q indexes the parameters subject to the penalty. The approximate minimization sign reflects the nonconvex training problem.

GKX use mini-batch Adam, which adjusts parameter step sizes using gradient-moment estimates. Their implementation also includes:

- **Early stopping:** retain the best validation checkpoint; patience allows several checks without improvement.
- **Batch normalization:** normalize activations, with learned scale and location parameters. The appendix applies it after ReLU; the compact slide formula suppresses this step.
- **Ensembles:** average forecasts from networks initialized with different random seeds.

NN1–NN5 and the reported training settings

Model	Numbers of neurons in successive hidden layers
NN1	32
NN2	32, 16
NN3	32, 16, 8
NN4	32, 16, 8, 4
NN5	32, 16, 8, 4, 2

The architectures are fixed for comparison. Table A.5 reports $\lambda \in (10^{-5}, 10^{-3})$, learning rates $\{0.001, 0.01\}$, batch size 10,000, at most 100 epochs, patience five, and an ensemble of ten networks. Greater depth increases flexibility but need not improve forecasts in weak-signal data. *Source: [1, Sec. 1.7]; [2, Appendix B.3; Table A.5].*

9 Performance Evaluation: Out-of-Sample R^2

Presentation reference: Performance Evaluation; Tables 1–2.

Historical average benchmark

For an aggregate return series, the slide uses

$$R^2_{\text{OOS}} = 1 - \frac{\sum_{t\in\mathcal{T}_{\text{test}}}(r_{t+1} - \widehat{r}_{t+1})^2}{\sum_{t\in\mathcal{T}_{\text{test}}}(r_{t+1} - \overline{r}_t)^2}, \tag{9.1}$$

where $\overline{r}_t$ uses only historical returns available at forecast time t. It is not the full-test-sample mean.

GKX's zero excess return benchmark

For individual stocks, GKX use

$$R^2_{\text{OOS}} = 1 - \frac{\sum_{(i,t)\in\mathcal{T}_{\text{test}}}(r_{i,t+1} - \widehat{r}_{i,t+1})^2}{\sum_{(i,t)\in\mathcal{T}_{\text{test}}} r^2_{i,t+1}}. \tag{9.2}$$

Here the test set contains stock-month observations. Historical individual-stock means can be sufficiently noisy that forecasting zero is a stronger benchmark.

Positive R^2_{OOS} means smaller total squared forecast error than the stated benchmark; a negative value means larger error. For example, 0.40% means a fraction 0.004 reduction in that benchmark's squared-error sum.

The statistic pools squared errors before taking their ratio. It is not an average of separately calculated monthly or stock-level R^2 values. The top/bottom 1,000 rows evaluate size subsamples selected each month by market equity, using forecasts from the full fitted model. Annual-return forecasting results use a different target horizon; they are not obtained by multiplying monthly R^2 by twelve.

Economic meaning: the Campbell–Thompson illustration

In the unconstrained mean–variance timing illustration underlying the slide, with constant conditional variance and both predictive R^2 and SR^2 small,

$$\text{relative increase in expected portfolio excess return} \approx \frac{R^2_{\text{OOS}}}{SR^2}. \tag{9.3}$$

SR is the passive asset's Sharpe ratio at the same frequency. Using the slide's rounded monthly inputs, $SR^2 \simeq 0.012$ and $R^2_{\text{OOS}} = 0.0043$, gives $0.0043/0.012 \simeq 36\%$.

This is a relative, model-based investment illustration, not a 36-percentage-point market return. It does not mechanically convert the pooled zero-benchmark stock R^2 into an attainable portfolio gain. *Source: [1, Sec. 1.8; Tables 1–2]; [3].*

10 Selected Model Complexity and Stock Characteristic Importance

Presentation reference: Figure 3; Figures 4(a)–4(b).

What Figure 3 measures

Method	Complexity measure in the paper's figure
ENet+H	Number of features with nonzero coefficients
PCR / PLS	Number of retained components
GLM+H	Selected features, whose spline coefficient groups are active
RF	Average tree depth
GBRT+H	Number of distinct characteristics entering the tree ensemble

These are different measures with different units. In particular, the GBRT panel does not show its number of trees. NN1–NN5 are fixed architectures, so the paper does not add a neural-network complexity panel to Figure 3.

Zero-out importance underlying Figure 4

Hold the fitted model fixed. Let $z_{i,t}^{(-j)}$ denote an input with predictor j set to zero. On a training sample, compare the original fit with

$$\widehat{r}_{i,t+1}^{(-j)} = g(z_{i,t}^{(-j)}; \widehat{\theta}).$$

The reduction in predictive fit is

$$VI_j = R^2_{\text{train}}(\widehat{g}) - R^2_{\text{train}}(\widehat{g}^{(-j)}) \tag{10.1}$$

$$= \frac{\sum_{\mathcal{D}_{\text{train}}} \left[(r_{i,t+1} - \widehat{r}_{i,t+1}^{(-j)})^2 - (r_{i,t+1} - \widehat{r}_{i,t+1})^2 \right]}{\sum_{\mathcal{D}_{\text{train}}} r_{i,t+1}^2}. \tag{10.2}$$

A large positive value means the fitted model relies heavily on that input. This is a perturbation of a fitted model, not a refit after deleting a variable.

GKX average importance across annual training samples and normalize within a model to sum to one for the figure. The chart reports relative contributions within that model, rather than comparable coefficient magnitudes across models.

Interpretation

The figure summarizes characteristic-level importance. Because the 920 constructed inputs include interactions, a characteristic label and a single input coordinate need not be identical; a numerical implementation must specify the mapping.

Importance measures predictive reliance, not a causal effect or a sign of the return relationship. Correlated predictors can share information, and setting an input to zero may sometimes improve fit, yielding a negative unnormalized importance. The figures' use of training fits should be distinguished from the test-sample forecast evaluation. *Source: [1, Sec. 1.9; Sec. 2.3; Figs. 3–4].*

11 Sorted Portfolios

Presentation reference: Sorted Portfolios; Table 7; Figure 9.

Monthly formation

At the end of month t, use information available by that date to obtain $\widehat{r}_{i,t+1}$. Sort stocks into ten deciles $D_{d,t}$, from lowest to highest predicted excess return. Reconstitute the portfolios each month.

Within a decile, value weights use market capitalization known at formation:

$$w_{i,d,t} = \frac{\mathrm{ME}_{i,t}}{\sum_{j \in D_{d,t}} \mathrm{ME}_{j,t}}, \qquad i \in D_{d,t}. \tag{11.1}$$

Predicted and realized portfolio excess returns are

$$\widehat{r}_{d,t+1} = \sum_{i \in D_{d,t}} w_{i,d,t} \widehat{r}_{i,t+1}, \tag{11.2}$$

$$r_{d,t+1} = \sum_{i \in D_{d,t}} w_{i,d,t} r_{i,t+1}. \tag{11.3}$$

The prediction and the realized return use the same formation-date holdings.

The high-minus-low strategy

Buy the highest-forecast decile and short the lowest:

$$r_{H-L,t+1} = r_{10,t+1} - r_{1,t+1}. \tag{11.4}$$

With each decile normalized to unit weight, this is a dollar-long/dollar-short, zero-net-investment spread before trading and financing costs. Its realized volatility must be calculated from the spread return series; it is not the difference between the two legs' volatilities.

Reading the Table 7 columns

Column	Meaning
Pred	Time average of predicted monthly portfolio returns
Avg	Time average of realized monthly portfolio returns
SD	Standard deviation of realized monthly portfolio returns
SR	Annualized Sharpe ratio, conventionally $\sqrt{12}\,\mathrm{Avg}/\mathrm{SD}$

Returns and standard deviations must use consistent units. A small stock-level R^2 can coexist with a useful portfolio strategy because the strategy depends on ranking, weighting, and diversification, rather than reproducing every realized stock return.

Figure 9

The figure reports cumulative log returns for the long and short positions separately, with solid and dashed lines. Its vertical scale should not be read as a simple cumulative percentage return, and the short position should not be interpreted as an additional long holding of the bottom decile. *Source: [1, Sec. 2.4.2; Table 7; Fig. 9].*

12 Portfolio Risk, Turnover, and Factor-Adjusted Performance

Presentation reference: Table 8.

Drawdowns and maximum monthly loss

Let Y_t be cumulative log strategy return. The paper displays

$$\text{MaxDD}_{\log} = \max_{0 \le t_1 \le t_2 \le T} (Y_{t_1} - Y_{t_2}). \tag{12.1}$$

This is a peak-to-trough loss measured in log-return units. A log drawdown D corresponds to a wealth decline $1 - \exp(-D)$; this differs from labelling D itself as a conventional wealth-loss fraction.

Maximum one-month loss describes the most negative realized monthly return. The table presents losses as positive magnitudes.

Turnover

Let $\widetilde{w}_{i,t+1}$ be the holding weight after returns have changed portfolio values, but before the next rebalance. Let $w_{i,t+1}$ be the new target weight. The paper's turnover convention sums absolute trading changes:

$$\text{Turnover} = \frac{1}{T} \sum_{t=1}^{T} \sum_{i} |w_{i,t+1} - \widetilde{w}_{i,t+1}|. \tag{12.2}$$

The drifted weights account for realized holding returns. This convention does not insert a factor of one-half. Long and short holdings can generate turnover above 100%. Turnover indicates trading intensity; the table's reported portfolio returns should not be described as net of transaction costs.

FF5+Mom alpha

For a high-minus-low strategy, estimate

$$\begin{aligned} r_{H-L,t+1} = \alpha &+ \beta_M \text{MKT}_{t+1} + \beta_S \text{SMB}_{t+1} + \beta_H \text{HML}_{t+1} \\ &+ \beta_R \text{RMW}_{t+1} + \beta_C \text{CMA}_{t+1} + \beta_U \text{Mom}_{t+1} + u_{t+1}. \end{aligned} \tag{12.3}$$

These are market excess return, size, value, profitability, investment, and momentum factors. Alpha is the average return unexplained by this factor model, expressed monthly in Table 8. Its statistic is

$$t(\widehat{\alpha}) = \widehat{\alpha} / \operatorname{se}(\widehat{\alpha}).$$

The annualized information ratio uses residual risk:

$$\text{IR} = \sqrt{12}\, \frac{\widehat{\alpha}}{\widehat{\sigma}_u}. \tag{12.4}$$

The regression R^2 measures explained time-series return variation. It is distinct from the out-of-sample forecasting R^2 in Section 9. Positive alpha is relative to the chosen factor benchmark and does not by itself identify mispricing. *Source: [1, Sec. 2.4.2; Table 8].*

13 Comparison and Further Reading

What changes across the forecasting methods?

Method	Forecasting structure	Main control on complexity
OLS / OLS-3	Linear in the supplied predictors	OLS-3 fixes a small input set
ENet	Linear in the supplied predictors	Coefficient selection and shrinkage
PCR / PLS	Linear combinations of predictors	Number of retained components
GLM	Additive nonlinear spline effects	Group penalty and spline specification
RF	Conditional partitions of predictor space	Tree depth, random feature subsets, averaging
GBRT	Sequential sum of trees	Tree depth, boosting steps, learning rate
NN1–NN5	Layers of nonlinear transformations	Architecture, penalty, early stopping, ensembles

All methods use the same forecast target. Their differences concern approximation, estimation, and regularization.

Questions the comparisons help answer

Why can regularization improve a forecast while worsening training fit? It can reduce the contribution of estimated noise, improving performance on later observations.

Why can nonlinear interactions matter? A predictor may have different associations with returns in different regions of predictor space. Trees and neural networks can learn such conditional patterns beyond the prespecified interactions in z.

Why need the best forecasting R^2 and best Sharpe ratio not coincide? Pooled squared error and a weighted decile strategy emphasize different aspects of a forecast. Rankings, extreme portfolios, weighting, and return covariance all matter for the strategy.

What do improved forecasts establish? They improve measurement of conditional expected returns. Distinguishing risk compensation from mispricing requires additional economic structure.

References

[1] Gu, S., B. Kelly, and D. Xiu. 2020. *Empirical Asset Pricing via Machine Learning*. The Review of Financial Studies 33(5): 2223–2273. doi:10.1093/rfs/hhaa009.

[2] Gu, S., B. Kelly, and D. Xiu. 2020. *Internet Appendix to Empirical Asset Pricing via Machine Learning*. Especially Algorithms 1–7, Appendix D, and Table A.5.

[3] Campbell, J. Y., and S. B. Thompson. 2008. *Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average?* The Review of Financial Studies 21(4): 1509–1531. doi:10.1093/rfs/hhm055.